Guide de déploiement de la plateforme CIR

Ce document est destiné à la personne ou à l'équipe qui met le site en ligne. Il est **indépendant de l'hébergeur** : le back-end Django et les deux applications React peuvent être installés chez n'importe quel fournisseur (serveur, conteneurs, plateforme cloud) et sous n'importe quel nom de domaine. Il liste tout ce qu'il faut préparer, installer et renseigner.

1. Ce qu'il faut mettre en ligne

Élément	Rôle	Technologie	Où dans le dépôt
Back-end (API)	API REST et WebSocket, règles métier, administration technique.	Python, Django 5.2, serveur ASGI Daphne	`backend/`
Worker	Même code que le back-end, autre commande : envoie les e-mails, expire les paiements, purge les données anciennes.	Python (même image que le back-end)	`backend/`
Site public	Vitrine, catalogue, commande, messagerie, assistant.	Fichiers statiques (React, Vite)	`frontend/site/`
Tableau de bord	Application de l'équipe, jamais liée depuis le site public.	Fichiers statiques (React, Vite)	`frontend/tableau-de-bord/`
Base de données	Toutes les données de la plateforme.	PostgreSQL 16 (ou plus récent)	service à créer
Redis	Cache et canal WebSocket partagé. Obligatoire dès qu'il y a plus d'un processus (back-end + worker).	Redis 6 ou plus récent	service à créer
Stockage de fichiers	Deux dossiers durables : images et vidéos publiques ; pièces jointes privées de la messagerie.	disque persistant	à monter (section 3)

Adresses à prévoir

Trois adresses web en **HTTPS**, de votre choix : une pour le **site public**, une pour le **tableau de bord**, une pour l'**API**. Ce peuvent être trois sous-domaines d'un même domaine ou trois domaines différents. Dans la suite, on les appelle **ADRESSE_SITE**, **ADRESSE_TABLEAU** et **ADRESSE_API** (exemple : https://exemple.org, https://admin.exemple.org, https://api.exemple.org). Aucun nom de domaine ni hébergeur n'est imposé par le code.

Outils nécessaires

Back-end : **Python 3.12 ou plus récent** et `pip`. Front-end : **Node.js 22** et `npm` (uniquement pour construire les fichiers ; ensuite, un simple serveur de fichiers statiques suffit). Un serveur web ou un reverse proxy qui gère le **HTTPS** et le **WebSocket**.

Les quatre API externes

Service	Sert à	Variables à renseigner (back-end)	Obligatoire ?

Gmail API (Google)	Envoyer les e-mails : confirmations, notifications, reçus.	`GMAIL_CLIENT_ID`
		`GMAIL_CLIENT_SECRET`
		`GMAIL_REFRESH_TOKEN`
		`GMAIL_EXPEDITEUR`
			Recommandé. Sans lui, les e-mails restent en attente.

CamPay	Payer par MTN Mobile Money et Orange Money.	`CAMPAY_USERNAME`
		`CAMPAY_PASSWORD`
		`CAMPAY_WEBHOOK_KEY`
		`CAMPAY_BASE_URL`
			Non. Sans lui, l'équipe enregistre les règlements hors ligne.

Jitsi as a Service (8x8 JaaS)	Appels audio et vidéo client ↔ équipe.	`JAAS_APP_ID`
		`JAAS_KEY_ID`
		`JAAS_PRIVATE_KEY`
		`JAAS_WEBHOOK_SECRET`
			Non. Sans lui, les boutons d'appel sont masqués.

Gemini API (Google)	Assistant IA du site et brouillons pour l'équipe.	`GEMINI_API_KEY`	**Non.** Sans lui, la bulle d'assistant est masquée.

Les clés d'API se saisissent **uniquement comme variables d'environnement** du back-end et du worker (ou dans un fichier `.env` non versionné), jamais dans le code, jamais sur GitHub, jamais dans un message. Les coûts et les conditions de chaque service

sont à vérifier sur leurs sites officiels. Pour Gmail, l'application Google doit être publiée en « Production » (sinon le jeton expire au bout de 7 jours). Dans CamPay, l'adresse du webhook à saisir est `ADRESSE_API/api/paiements/webhooks/campay/`.

2. Installer le back-end Django

Le dossier `backend/` est la racine du back-end. Les mêmes variables d'environnement servent au back-end **et** au worker.

Commandes

```
pip install -r requirements.txt
python manage.py migrate --noinput # à chaque mise à jour, avant le démarrage
python manage.py collectstatic --noinput # fichiers statiques (servis par WhiteNoise)
daphne -b 0.0.0.0 -p 8000 config.asgi:application # processus web (HTTP + WebSocket)
python manage.py envoyer_emails --boucle # processus worker (à lancer à part, en continu)
```

Le programme doit écouter sur le port que votre hébergeur ou votre reverse proxy attend. Test de santé à configurer : `GET /api/sante/` doit répondre « ok » (sans authentification). Pour utiliser Daphne, il faut le serveur ASGI : n'utilisez pas un serveur WSGI seul, la messagerie en temps réel ne marcherait pas.

Variables obligatoires

Variable	Quoi mettre	Votre valeur
`DJANGO_SETTINGS_MODULE`	`config.settings.prod`	
`DJANGO_SECRET_KEY`	**Secret.** 50 caractères aléatoires au minimum (sinon l'application refuse de démarrer). À générer : `python -c "import secrets; print(secrets.token_urlsafe(64))"`	
`DJANGO_ALLOWED_HOSTS`	Nom d'hôte de ADRESSE_API, sans https:// (plusieurs valeurs séparées par des virgules). Ajouter aussi tout nom d'hôte utilisé par la sonde de santé de votre hébergeur.	
`DATABASE_URL`	Adresse PostgreSQL : `postgres://utilisateur:mot_de_passe@hote:5432/nom_base`	
`REDIS_URL`	Adresse Redis : `redis://hote:6379/0` (indispensable avec le worker)	
`CORS_ALLOWED_ORIGINS`	**À renseigner.** ADRESSE_SITE et ADRESSE_TABLEAU, avec https://, séparées par des virgules. Le défaut du code vise le domaine d'origine du projet, pas le vôtre.	
`DJANGO_CSRF_TRUSTED_ORIGINS`	**À renseigner.** ADRESSE_SITE, ADRESSE_TABLEAU et ADRESSE_API, avec https://, séparées par des virgules.	
`SITE_URL`	ADRESSE_SITE (liens dans les e-mails)	
`ADMIN_URL_PUBLIQUE`	ADRESSE_TABLEAU (liens dans les e-mails de l'équipe)	
`API_URL_PUBLIQUE`	ADRESSE_API (lien de désabonnement, webhooks)	
`DJANGO_MEDIA_ROOT`	Dossier **persistant** des images et vidéos, ex. `/donnees/media`	
`DJANGO_PRIVATE_ROOT`	Dossier **persistant et privé** des pièces jointes, ex. `/donnees/prive`. Jamais exposé sur le web.	
`NOTIFICATIONS_ENVOI_IMMEDIAT`	`0` en production : les e-mails sont envoyés par le worker	
`DEFAULT_FROM_EMAIL`	Expéditeur affiché, ex. `CIR <no-reply@votre-domaine>`	
`DJANGO_ADMIN_URL`	**À choisir.** Adresse cachée de l'administration technique de Django, terminée par une barre oblique, ex. `gestion-interne-x7/`	

Variables facultatives

Variable	Rôle et valeur par défaut
`DJANGO_NOMBRE_PROXIES`	Nombre de proxys de confiance devant l'application (adresse du client lue dans X-Forwarded-For). Défaut 1 ; mettre 0 sans proxy, ou le nombre réel d'intermédiaires.
`DJANGO_SECURE_SSL_REDIRECT`	Défaut 1 : redirige le HTTP vers le HTTPS (sauf `/api/sante/`). Le proxy doit envoyer l'en-tête `X-Forwarded-Proto: https`, sinon boucle de redirection.
`DJANGO_SERVIR_MEDIAS`	Défaut 1 : Django sert `/media/`. Mettre 0 si votre serveur web sert ce dossier lui-même.
`JWT_ACCES_MINUTES` `JWT_RAFRAICHISSEMENT_JOURS`	30 minutes et 7 jours : durée des connexions.
`GMAIL_LIMITE_JOUR` `JAAS_LIMITE_GRATUITE`	400 e-mails par jour ; 25 utilisateurs d'appel actifs par mois (alertes).
`PAIEMENT_MONTANT_MINIMUM` `PAIEMENT_EXPIRATION_MINUTES`	100 FCFA ; 15 minutes avant l'expiration d'un paiement en attente.
`ASSISTANT_LIMITE_JOUR` `ASSISTANT_RETENTION_JOURS`	1000 messages d'assistant par jour pour tout le site ; conversations gardées 90 jours.
`GEMINI_MODELE_ASSISTANT` `GEMINI_MODELE_FOND`	`gemini-2.5-flash` : à changer si Google renomme le modèle.

Fichiers, worker et premières données

Deux dossiers durables. Ils doivent survivre aux redéploiements (disque persistant, volume, stockage monté). Le dossier public (`DJANGO_MEDIA_ROOT`) contient les images et vidéos ; le dossier privé (`DJANGO_PRIVATE_ROOT`) contient les pièces jointes de la messagerie et ne doit **jamais** être servi directement par le serveur web.

Worker. Même code, mêmes variables, sans adresse publique. Il envoie les e-mails, expire les paiements restés en attente et purge les données anciennes. Sans lui, aucun e-mail ne part. Il doit voir le **même dossier privé** que le back-end (même machine ou disque partagé) pour pouvoir purger les pièces jointes ; si ce n'est pas possible chez votre hébergeur, prévenez l'équipe de développement.

WebSocket. La messagerie et les appels passent par `wss://ADRESSE_API/ws/`. Le reverse proxy doit autoriser la mise à niveau `Upgrade: websocket` et ne pas couper les connexions inactives trop vite.

Données de départ (une seule fois, après la migration).

```
python manage.py charger_catalogue_reference # 9 familles et 16 produits, sans prix ni image
python manage.py importer_medias_reference # vidéo de fond, affiche, photos de la sphère, 6 images de produits
```

3. Construire et publier les deux applications React

Le site et le tableau de bord sont deux applications **statiques** séparées. Elles partagent le dossier `frontend/partage/` : la construction doit donc se faire dans une copie complète du dépôt (le dossier `frontend/` avec ses trois sous-dossiers).

```
# pour le site public, puis pareil dans frontend/tableau-de-bord
cd frontend/site
npm ci
VITE_API_URL=ADRESSE_API npm run build # produit le dossier dist/
```

À renseigner	Site public	Tableau de bord
Variable de construction	`VITE_API_URL` = ADRESSE_API (sans barre finale)	`VITE_API_URL` = ADRESSE_API (sans barre finale)
Dossier à publier	`frontend/site/dist/`	`frontend/tableau-de-bord/dist/`
Adresse de publication	ADRESSE_SITE	ADRESSE_TABLEAU (jamais liée depuis le site)

`VITE_API_URL` est lue à la construction : si l'adresse de l'API change, il faut reconstruire et republier les deux applications. Sans cette variable, les applications cherchent l'API sur `http://localhost:8000`.

Règles du serveur de fichiers statiques : (1) **toute adresse inconnue renvoie** `index.html` (application à page unique, sinon les liens directs donnent une erreur 404) ; (2) HTTPS obligatoire ; (3) cache long conseillé pour `/assets/` (noms de fichiers à empreinte), pas de cache pour `index.html`. Deux modèles prêts à l'emploi existent

aussi : `frontend/site/Dockerfile` et `frontend/tableau-de-bord/Dockerfile` (contexte de construction = racine du dépôt, argument `VITE_API_URL`).

4. Comptes à créer

Compte super-administrateur

À renseigner	Détail	Votre valeur
E-mail	Adresse du propriétaire : c'est l'identifiant de connexion.	
Nom complet	Affiché dans le tableau de bord.	
Mot de passe	**Secret : ne jamais l'écrire ici, ni dans un message, ni sur GitHub.** Règles : 8 caractères au minimum, pas un mot de passe courant, pas uniquement des chiffres, pas proche de l'e-mail ou du nom. Conseil : 14 caractères ou plus ; le garder dans un gestionnaire de mots de passe.	(à noter dans un gestionnaire)

Création : dans un terminal sur le back-end (dossier `backend/`, mêmes variables d'environnement), lancer `python manage.py createsuperuser` ; la commande demande l'e-mail, le nom complet, puis le mot de passe (deux fois). Connexion ensuite sur ADRESSE_TABLEAU. Le super-administrateur crée les comptes de l'équipe dans le menu « Équipe » (e-mail, nom, rôle, mot de passe provisoire). Seul lui voit les sommes d'argent des statistiques. L'administration technique de Django (`ADRESSE_API/DJANGO_ADMIN_URL`) est une porte de secours, distincte du tableau de bord.

Informations publiques à saisir dans le tableau de bord

Menu « Contenus du site ». Éléments connus : téléphones +237 6 99 81 16 35, +237 6 78 82 93 13, +237 6 56 43 58 46 ; WhatsApp +237 622 191 931 ; adresse : Total Nkolmbong, Douala. **À fournir par le propriétaire** : e-mail public, textes de présentation du partenaire Jiuye, et les prix (facultatifs : sans prix, le site affiche « Prix sur devis »). Aucun prix, témoignage ni nom de client n'est inventé.

Ordre conseillé de mise en ligne

N°	Étape
1	Créer PostgreSQL et Redis ; préparer les deux dossiers durables et choisir les trois adresses HTTPS.
2	Renseigner les variables du back-end (section 2), puis lancer `migrate`, `collectstatic` et démarrer Daphne. Vérifier `ADRESSE_API/api/sante/`.
3	Démarrer le worker avec les mêmes variables.
4	Créer le super-administrateur (section 4).
5	Charger le catalogue et les médias de départ (section 2, « Données de départ »).
6	Construire et publier le site, puis le tableau de bord avec `VITE_API_URL` (section 3).
7	Se connecter au tableau de bord, créer les comptes de l'équipe, compléter les contenus publics.
8	Gmail : créer le projet Google, publier l'application en « Production », saisir les variables, vérifier qu'un e-mail de test part.
9	JaaS : créer l'application, générer la paire de clés (la clé privée n'est affichée qu'une fois), saisir les variables.
10	CamPay : essayer d'abord avec `CAMPAY_BASE_URL=https://demo.campay.net/api`, puis passer en production et saisir l'adresse du webhook.
11	Gemini : créer la clé, saisir la variable ; prévoir le niveau payant si le trafic grandit.

Règles de sécurité

Aucune clé ni aucun mot de passe dans le code, dans un document ou sur GitHub (le fichier `.env` n'est jamais versionné). Si une clé a été exposée, la remplacer aussitôt chez le fournisseur. Utiliser des clés de démonstration pour les essais et des clés distinctes en production. Si le démarrage du back-end échoue : vérifier d'abord `DJANGO_SECRET_KEY` (50 caractères au moins), `DATABASE_URL`, puis les journaux du processus.